MILWAUKEE TOOL · ONE-KEY EMEA · DIGITAL PRODUCT

Weekend Challenge

Working Student Product Management — Full-Stack Developer (m/f/d) Milwaukee®
· Take-Home Assignment

Deadline: Share your GitHub repo link 1 day before our meeting**.** You will present your solution live to a mixed room. Keep your explanations simple — not everyone in the room is technical.

Hi there,

Here is your weekend challenge. Read everything before you start. If something feels unclear, make a decision, write it down, and keep going — that is exactly what this job is about.

THE CONTEXT

A technology company is developing new products and regularly sends physical sample units to different teams across several European countries so they can be tested and demonstrated before launch. Because these samples move between countries and teams, it's difficult to maintain a clear overview of where each item currently is or what its status might be.

Your challenge is to build a simple system that solves this problem: a central tool list that can assign items to different country accounts and keep everything synchronized.

Here's the text:

For this challenge, you will **build your own mock API server**.
It does *not* need to be complex — the goal is simply to have a working back-end that your front-end can communicate with.

You are free to choose any technology stack you like.
And of course — feel free to use **AI tools** to support yourself throughout the task (for coding, structuring data, generating test sets, etc.). AI assistance is allowed and encouraged, as long as you understand what your code is doing.

To help you explain this to a non-technical audience, use this table as a reference:

Technical Term	What it means in plain language
API	A messenger that takes a request from one system and delivers the answer from another — like a waiter between two kitchens
Warehouse Account	The main storage room where all tool samples live before being sent out to countries
Demo Account	The testing room in a specific country — tools go here for demonstrations and field trials
Sync	Automatically keeping both lists up to date so they always match
Audit Trail	A logbook — a record of every movement, who triggered it, and when
Authentication	A key or password that proves your app is allowed to talk to the system

YOUR TASKS

1 Read the Mock API Specification

Your mock API should describe (in OpenAPI / Swagger format) the three required endpoints that your back-end will implement:

`/auth/token`	Get an access token using a client ID and secret
`/tools`	Get the list of tools in the Warehouse account
`/transfer`	Move a selected tool from the Warehouse to a Demo Account

Show us that you can read the spec, understand what each endpoint does, and know what fields are required and how authentication works. You do not need to explain this technically in your presentation — just show that your app uses it correctly.

2 Build a Small Back-End (Python or Node.js)

Your back-end must:

- ✔ Authenticate — call /auth/token and use the returned token for subsequent requests
- ✔ Fetch tools — retrieve the tool list from the Warehouse
- ✔ Transfer tools — move selected tools to the correct country Demo Account
- ✔ Log every action — what happened, when, and whether it succeeded or failed

Error messages must be simple and human-friendly. No technical language — write them as if a non-developer will read them.

- Wrong login -> "Your login details are incorrect. Please check and try again."
- Expired session -> "Your session has expired. Please log in again."
- No permission -> "You do not have permission to do this."
- Server problem -> "Something went wrong. Please try again in a moment."

These messages must be available in at least two languages. English is required. Choose any second language that feels natural to you — German, French, Spanish, Italian, or other. Your back-end should accept a language parameter and respond accordingly.

3 Build a Simple Front-End (Any framework is fine)

One screen. Keep it clean and easy to use — imagine a sales manager, not a developer, is using this every day.

- ✔ Show the list of tools from the Warehouse with their current status
- ✔ Let the user select tools using checkboxes
- ✔ A button to move selected tools to a Demo Account
- ✔ Show clear success or error messages after each action
- ✔ A language switcher — English plus your chosen second language

The multilingual requirement applies to everything visible: buttons, labels, messages, and titles. Structure your translations properly — do not hard-code strings scattered through the code.

4 Prepare a 10 Minute Presentation

Your audience includes people who have never written a line of code and do not know what an API is. Speak to them, not to developers.

Cover these points:

- ✔ What problem does this solve — in plain, everyday words
- ✔ How your solution works — use real-life comparisons from the table above
- ✔ A short live demo of your working app
- ✔ One or two things you would improve with more time

> **Presentation tip:**
> *Before you open your laptop, be ready to answer this question in two sentences: what did you build, and why does it matter? If you can answer that clearly, the rest of the presentation will follow naturally.*

WHAT TO SUBMIT

Send us a link to your GitHub repository. That is all we need.

Your repo should include:

- A /backend folder and a /frontend folder
- A README.md explaining how to run it locally in under 5 minutes
- A short note at the end of the README: what would you do differently with more time

Optional but appreciated: a Postman collection showing your API calls.

WHAT WE ARE LOOKING AT

We are not looking for a perfect solution. We are looking at:

- How you break down a vague problem and make decisions
- Whether your front-end and back-end actually work together
- Code that is clean enough for someone else to read and pick up
- A multilingual setup that is structured, not just patched on
- How clearly you explain your work to a non-technical room

Good luck. We are looking forward to seeing what you build.